Adham Ashraf Saeed

Assignment 3 (SQL)

Load the provided dataset into your database

```
CREATE DATABASE MoviesDB;

USE MoviesDB;
```

- Open the provided link and build up your database.

Data Source

Questions

1. Display all movies

```
SELECT *
FROM   movies;
```

2. Display all movie ratings.

```
SELECT *
FROM   ratings;
```

3. Display all movie IDs that appear in the ratings table.

```
SELECT DISTINCT movie_id
FROM   ratings;
```

4. Display movie titles along with their ratings.

```
SELECT M.title,
       R.rating
FROM   movies AS M
       INNER JOIN
       ratings AS R
       ON R.movie_id = M.movie_id;
```

5. Display movie titles and their budgets for movies that have ratings.

```
SELECT DISTINCT M.title,
                M.budget
FROM   movies AS M
       INNER JOIN
       ratings AS R
       ON R.movie_id = M.movie_id;
```

Questions

6. Display movies that appear in both Movies and Credits tables.

```
SELECT DISTINCT M.movie_id,
                M.title
FROM   movies AS M
       INNER JOIN
       credits AS C
       ON C.movie_id = M.movie_id;
```

7. Display all movies and their ratings (include movies with no ratings).

```
SELECT M.title,
       R.rating
FROM   movies AS M
       LEFT JOIN
       ratings AS R
       ON R.movie_id = M.movie_id;
```

8. Display movies that have no ratings.

```
SELECT M.movie_id,
       M.title
FROM   movies AS M
       LEFT JOIN
       ratings AS R
       ON R.movie_id = M.movie_id
WHERE  R.movie_id IS NULL;
```

9. Display movie title and average rating (include movies with no ratings).

```
SELECT   M.title,
         AVG(R.rating) AS [Avg Rating]
FROM     movies AS M
         LEFT JOIN
         ratings AS R
         ON R.movie_id = M.movie_id
GROUP BY M.title;
```

Questions

10. Display all movie IDs from Movies and Ratings, matched where possible.

```
SELECT M.movie_id AS M_id,
       R.movie_id AS R_id
FROM   movies AS M
       FULL OUTER JOIN
       ratings AS R
       ON M.movie_id = R.movie_id;
```

11. Identify movies that exist in Movies but not in Ratings, and movies that exist in Ratings but not in Movies.

```
SELECT movie_id
FROM   movies
EXCEPT
SELECT movie_id
FROM   ratings;
```

```
SELECT movie_id
FROM   ratings
EXCEPT
SELECT movie_id
FROM   movies;
```

12. Display a list of all movie IDs from Movies and Ratings without duplicates.

```
SELECT movie_id
FROM   ratings
UNION
SELECT movie_id
FROM   movies;
```

13. Display a list of all movie IDs from Movies and Ratings including duplicates.

```
SELECT movie_id
FROM   ratings
UNION ALL
SELECT movie_id
FROM   movies;
```

Questions

14. Display all movie titles and all user IDs in a single column.

```
SELECT title
FROM   movies
UNION ALL
SELECT CONVERT (VARCHAR, [user_id])
FROM   ratings;
```

15. Display movies that have ratings from more than 10 different users.

```
SELECT   M.title,
         COUNT(DISTINCT R.[user_id]) AS [Total Users]
FROM     movies AS M
         INNER JOIN
         ratings AS R
         ON M.movie_id = R.movie_id
GROUP BY M.title
HAVING   COUNT(DISTINCT R.[user_id]) > 10;
```

16. Display movies whose average rating is higher than the overall average rating.

```
SELECT   M.title,
         AVG(R.rating) AS [Avg Rating]
FROM     movies AS M
         INNER JOIN
         ratings AS R
         ON M.movie_id = R.movie_id
GROUP BY M.title
HAVING   AVG(R.rating) > (SELECT AVG(rating)
                          FROM   ratings);
```

17. Display movies that appear in Credits but have no ratings.

```
SELECT DISTINCT M.movie_id,
                M.title
FROM   movies AS M
       INNER JOIN
       credits AS C
       ON C.movie_id = M.movie_id
       LEFT JOIN
       ratings AS R
       ON R.movie_id = M.movie_id
WHERE  R.movie_id IS NULL;
```